

TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP
KHOA ĐIỆN TỬ
BỘ MÔN: CÔNG NGHỆ THÔNG TIN



BÀI TIỂU LUẬN

MÔN HỌC

ĐẢM BẢO CHẤT LƯỢNG VÀ KIỂM THỬ PHẦN MỀM

NGÀNH : KỸ THUẬT MÁY TÍNH

HỆ : ĐẠI HỌC CHÍNH QUY

Họ và tên sinh viên : Nguyễn Văn Thứ

Mã số sinh viên : K225480106062

Lớp : K58KTP

Giáo viên hướng dẫn : TS. Nguyễn Tuấn Linh

THÁI NGUYÊN – 2026

Thái Nguyên, ngày 01 tháng 06 năm 2026

PHIẾU GIAO ĐỀ TÀI

Họ tên sinh viên: Nguyễn Văn Thứ

MSSV: K225480106062

Ngành: Kỹ thuật máy tính

Lớp: K58KTP

Giáo viên hướng dẫn: TS. Nguyễn Tuấn Linh

Ngày giao đề: 01/06/2026

Ngày hoàn thành: 25/06/2026

Tên đề tài: Tự động refactoring bằng compiler generator/Roslyn/ANTLR

1. Yêu cầu:

- Mô tả cách công cụ phân tích cú pháp hỗ trợ chèn, sửa, xóa mã để refactor tự động và nâng testability.

2. Kết quả:

- Báo cáo theo mẫu của Khoa và bộ môn
- Slide báo cáo, video
- Chương trình mô phỏng

BM. CÔNG NGHỆ THÔNG TIN

(Ký và ghi rõ họ tên)

GIÁO VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

Thái Nguyên, ngày 25 tháng 06 năm 2026

PHIẾU GHI ĐIỂM

Sinh viên: Nguyễn Văn Thứ

MSSV: K225480106062

Lớp: K58KTP

GVHD: TS. Nguyễn Tuấn Linh

Tên đề tài: Tự động refactoring bằng compiler generator/Roslyn/ANTLR

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....
.....
.....
.....
.....

Xếp loại: Điểm:

Thái Nguyên, ngày tháng năm 2026

GIÁO VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

MỤC LỤC

MỤC LỤC	3
DANH MỤC TỪ VIẾT TẮT	4
LỜI CẢM ƠN.....	5
LỜI NÓI ĐẦU	6
CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI	7
1.1. Lý do chọn đề tài.....	7
1.2. Mục tiêu nghiên cứu.....	8
1.3. Phạm vi nghiên cứu.....	8
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN	10
2.1. Định nghĩa và các khái niệm cốt lõi.....	10
2.3. Sự liên hệ với chất lượng phần mềm	13
CHƯƠNG 3: VẬN DỤNG VÀ THỰC HÀNH.....	15
3.1. Mô tả bài toán ngữ cảnh thực tế.....	15
3.2. Áp dụng giải pháp	16
3.3. Kết quả và đánh giá.....	17
CHƯƠNG 4: KẾT LUẬN VÀ KIẾN NGHỊ	20
4.1. Tổng kết nội dung	20
4.2. Hạn chế và giải pháp	20
4.3. Định hướng mở rộng.....	21
TÀI LIỆU THAM KHẢO	23

DANH MỤC TỪ VIẾT TẮT

1. AST (Abstract Syntax Tree): Cây cú pháp trừu tượng
2. DI (Dependency Injection): Tiêm phụ thuộc
3. QA/QC (Quality Assurance / Quality Control): Đảm bảo chất lượng - Kiểm soát chất lượng phần mềm.
4. TDD (Test-Driven Development): Phát triển hướng kiểm
5. E2E (End-to-End Testing): Kiểm thử hộp đen toàn trình hệ thống
6. CI/CD (Continuous Integration / Continuous Deployment): Tích hợp liên tục và Triển khai liên tục.
7. API (Application Programming Interface): Giao diện lập trình ứng dụng
8. FSM (Finite State Machine): Máy trạng thái hữu hạn
9. ANTLR (ANother Tool for Language Recognition)

LỜI CẢM ƠN

Trước hết, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến Thầy giáo giảng viên TS. Nguyễn Tuấn Linh người phụ trách giảng dạy môn học Đảm bảo chất lượng và kiểm thử phần mềm. Nhờ có sự định hướng khoa học, những bài giảng tâm huyết và sự hướng dẫn tận tình của Thầy, em đã có cơ hội tiếp cận với các tư duy kiểm thử hiện đại, hiểu sâu hơn về tầm quan trọng của cấu trúc mã nguồn đối với chất lượng một hệ thống phần mềm.

Em cũng xin gửi lời cảm ơn đến các bạn trong lớp đã đồng hành, trao đổi, chia sẻ tài liệu và hỗ trợ nhóm em trong quá trình tìm hiểu, phân tích và triển khai nội dung. Sự hợp tác và tinh thần làm việc nhóm của các bạn đã góp phần rất lớn vào việc hoàn thành bài tập với kết quả tốt nhất có thể.

Mặc dù đã có nhiều cố gắng nhưng do kiến thức và kinh nghiệm còn hạn chế, bài làm chắc chắn không tránh khỏi những thiếu sót, bám sát tài liệu gốc và vận dụng vào thực tiễn, bài tiểu luận chắc chắn không tránh khỏi những thiếu sót thiếu khách quan. Em rất mong nhận được những lời phê bình, góp ý quý báu từ Thầy/Cô để kiến thức của bản thân được hoàn thiện hơn.

Em xin chân thành cảm ơn!

LỜI NÓI ĐẦU

Trong kỷ nguyên chuyển đổi số toàn cầu, phần mềm đã trở thành hạ tầng cốt lõi vận hành mọi khía cạnh của đời sống xã hội. Khi quy mô và độ phức tạp của hệ thống tăng lên theo cấp số nhân, bài toán kiểm thử và đảm bảo chất lượng phần mềm (QA) không còn đơn thuần là việc "nhấn nút" (button-pushing) trên giao diện, mà đã dịch chuyển sâu sắc thành một chiến lược quản trị rủi ro kỹ thuật toàn diện. Một trong những thách thức lớn nhất mà các kỹ sư chất lượng phải đối mặt ngày nay chính là "Nợ kỹ thuật" (Technical Debt) tích tụ trong mã nguồn cũ (Legacy Code). Việc lạm dụng các hàm tĩnh (Static Methods) hoặc mô hình Singleton kết nối trực tiếp đến tài nguyên cứng (như Cơ sở dữ liệu, API bên ngoài) đã biến mã nguồn thành một khối thắt nút chặt chẽ, triệt tiêu hoàn toàn khả năng kiểm thử cô lập (Unit Test).

Đứng trước rủi ro chất lượng đó, ngành công nghiệp phần mềm hiện đại đang hướng tới xu hướng "Shift-Left" (Dịch chuyển kiểm thử về bên trái) bằng cách tối ưu hóa cấu trúc mã nguồn ngay từ tầng biên dịch. Cuốn giáo trình "Software Testing Strategies" (2023) của Matthew Heusser và Michael Larsen đã chỉ ra rằng, để nâng cao chỉ số *Testability* (Khả năng kiểm thử), lập trình viên bắt buộc phải tạo ra các "*Seams*" (Điểm khớp nối / Biên phân tách logic) nhằm chèn các đối tượng giả lập (Mocks/Stubs). Tuy nhiên, việc rà soát và Refactoring (Tái cấu trúc) mã nguồn một cách thủ công thường tốn kém chi phí lớn, dễ gây sai sót mang tính chủ quan của con người và đối mặt với rủi ro phá vỡ logic cũ.

Vì vậy, việc tự động hóa quy trình phân tích và biến đổi mã nguồn ở tầng trình biên dịch (Compiler Level) bằng các công cụ mạnh mẽ như ANTLR và Roslyn trở thành một giải pháp mang tính đột phá. Đề tài: "Tự động refactoring bằng compiler generator/Roslyn/ANTLR để nâng cao Testability" được thực hiện nhằm nghiên cứu sâu cơ chế phân tích cú pháp tĩnh qua cây AST (Abstract Syntax Tree), từ đó thiết lập quy trình chèn, sửa, xóa mã tự động, chuyển đổi các cấu trúc "khó kiểm thử" (Static/Singleton) sang kiến trúc lỏng sử dụng Dependency Injection (DI).

Trọng tâm thực hành của tiểu luận sẽ tập trung xây dựng module chuyển đổi tự động dựa trên trình tạo trình biên dịch ANTLR áp dụng cho ngôn ngữ Java/Python. Qua đó, bài viết hướng tới mục tiêu chứng minh sức mạnh của tự động hóa tầng sâu trong việc cải thiện chỉ số Code Coverage (Độ phủ mã nguồn), tối ưu hóa quy trình kiểm thử và nâng cao chất lượng phần mềm tổng thể.

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1. Lý do chọn đề tài

Trong kỷ nguyên phát triển phần mềm hiện đại, xu hướng tích hợp liên tục và bàn giao liên tục (CI/CD) đòi hỏi tốc độ kiểm thử phải bắt kịp tiến độ phát triển mã nguồn. Cuốn giáo trình học thuật "Software Testing Strategies" (2023) của Matthew Heusser và Michael Larsen đã nhấn mạnh rằng, việc đảm bảo chất lượng phần mềm không còn là công đoạn độc lập ở cuối chu kỳ mà phải dịch chuyển mạnh mẽ sang bên trái (Shift-Left Testing) – tức là kiểm thử được thực hiện ngay từ những giai đoạn đầu tiên của quá trình lập trình. Để hiện thực hóa chiến lược này, vai trò của kiểm thử đơn vị (Unit Testing) và khả năng viết mã kiểm thử một cách dễ dàng, cô lập trở thành nhân tố quyết định tính sống còn của một hệ thống.

Tuy nhiên, một rào cản nghiêm trọng mà hầu hết các dự án phần mềm lâu năm hoặc các hệ thống có quy mô lớn gặp phải chính là sự tích tụ của "Nợ kỹ thuật" (Technical Debt) trong cấu trúc mã nguồn cũ (Legacy Code). Biểu hiện phổ biến và nguy hại nhất của loại nợ kỹ thuật này là việc lạm dụng các hàm tĩnh (Static Methods), biến tĩnh hoặc các mẫu thiết kế dạng Singleton gắn chặt (Hard-coded) trực tiếp với các tài nguyên vật lý như Cơ sở dữ liệu, API bên ngoài hoặc các file cấu hình hệ thống. Khi một đoạn mã nguồn không có Seams, việc viết Unit Test trở nên bất khả thi hoặc đòi hỏi chi phí thiết lập môi trường cực kỳ tốn kém, khiến chỉ số khả năng kiểm thử (Testability) của phần mềm bị kéo giảm nghiêm trọng.

Để giải quyết bài toán này, quy trình tái cấu trúc mã nguồn (Refactoring) là bắt buộc. Thế nhưng, nếu thực hiện Refactoring bằng phương pháp thủ công do kỹ sư QA hoặc lập trình viên tự rà soát và chỉnh sửa, dự án sẽ đối mặt với ba thách thức lớn:

- Tồn kém nguồn lực và thời gian: Việc duyệt qua hàng ngàn tệp tin chứa mã nguồn để tìm và sửa thủ công cấu trúc Static/Singleton là một gánh nặng chi phí lớn.
- Rủi ro sai sót chủ quan: Con người rất dễ bỏ sót các mối liên kết tiềm ẩn hoặc làm thay đổi logic nghiệp vụ cũ trong quá trình sửa đổi, dẫn đến lỗi biên dịch hoặc lỗi logic nghiêm trọng.
- Thiếu tính nhất quán: Quy chuẩn viết lại code dễ bị phân tán do thói quen của từng lập trình viên khác nhau.

Chính vì vậy, việc tự động hóa quy trình Refactoring ở tầng trình biên dịch (Compiler Level) bằng cách sử dụng các bộ tạo trình biên dịch (Compiler Generators) như ANTLR

hoặc các nền tảng phân tích mã nguồn mạnh mẽ như Roslyn đã trở thành một hướng đi đột phá và cấp thiết. Giải pháp này cho phép phân tích cú pháp mã nguồn thành Cây cú pháp trừu tượng (Abstract Syntax Tree - AST), tự động nhận diện chính xác các "Code Smell" (Mã nguồn xấu) dạng Static/Singleton và thực hiện các thao tác chèn, sửa, xóa các nút trên cây cú pháp để chuyển đổi chúng sang cấu trúc lỏng sử dụng Dependency Injection (DI) thông qua các Interface.

Xuất phát từ tầm quan trọng của chỉ số Testability đối với chất lượng phần mềm, cùng với nhu cầu tối ưu hóa nguồn lực thông qua tự động hóa tầng sâu, em đã quyết định lựa chọn nghiên cứu đề tài: "Tự động refactoring bằng compiler generator/Roslyn/ANTLR để nâng cao Testability".

1.2. Mục tiêu nghiên cứu

Đề tài này tập trung giải quyết bài toán nâng cao năng lực đảm bảo chất lượng phần mềm từ gốc rễ cấu trúc mã nguồn. Các mục tiêu cụ thể bao gồm:

- Mục tiêu lý thuyết: Nghiên cứu và làm rõ mối quan hệ biện chứng giữa cấu trúc hình học của mã nguồn với khả năng kiểm thử (Testability). Phân tích sâu cơ chế hoạt động của các công cụ phân tích tĩnh (Static Analysis) thông qua mô hình Cây cú pháp trừu tượng (AST). So sánh và đánh giá nguyên lý chèn, sửa, xóa mã nguồn tự động của hai nền tảng tiêu biểu: Microsoft Roslyn (hệ sinh thái .NET/C#) và ANTLR (hệ sinh thái đa ngôn ngữ Java/Python).
- Mục tiêu thực tiễn (Yêu cầu chính): Thiết kế và mô tả chi tiết cách thức một công cụ phân tích cú pháp tự động can thiệp vào mã nguồn để thực hiện các hành vi Refactor tự động mà không làm thay đổi logic chức năng bên ngoài (Behavior-preserving).
- Mục tiêu thực hành ứng dụng: Xây dựng một module thực nghiệm cụ thể sử dụng ANTLR cho ngôn ngữ Java/Python nhằm tự động hóa việc phát hiện cấu trúc gọi hàm Static khó test, tự động viết lại (Rewrite) mã nguồn đó sang mô hình Dependency Injection dựa trên Interface, tạo ra các điểm khớp nối (Seams) giúp lập trình viên có thể dễ dàng viết Unit Test bằng Mock Object, nâng cao chỉ số Code Coverage (Độ phủ mã nguồn) của toàn bộ hệ thống.

1.3. Phạm vi nghiên cứu

Để đảm bảo tính tập trung và độ sâu của một bài tiểu luận học thuật, phạm vi nghiên cứu của đề tài được giới hạn rõ ràng trong các biên sau:

- Về khái niệm và lý thuyết: Đề tài tập trung nghiên cứu khái niệm Testability và cấu trúc Seams dưới góc nhìn của lập trình viên (Programmer-Facing Testing).

Không mở rộng sang các tầng kiểm thử phi chức năng khác như kiểm thử hiệu năng (Performance Testing) hay kiểm thử bảo mật (Security Testing) ở cấp độ hệ thống lớn.

- Về dạng mã nguồn xấu (Code Smell) cần Refactor: Giới hạn duy nhất ở bài toán chuyển đổi cấu trúc phụ thuộc cứng (Hard-coded) của các hàm tĩnh (Static Methods) hoặc cấu trúc Singleton kết nối tài nguyên sang cấu trúc thiết kế lỏng (Loose Coupling) thông qua Interface và Dependency Injection. Không xử lý các dạng Code Smell khác như mã nguồn trùng lặp (Duplicate Code) hay lớp quá lớn (Large Class).
- Về công nghệ và công cụ: Ở phần cơ sở lý thuyết, phạm vi nghiên cứu bao gồm cả nền tảng Roslyn (dành cho ngôn ngữ C#) và bộ tạo trình biên dịch ANTLR. Đề tài giới hạn phạm vi triển khai mã nguồn minh họa và xây dựng ngữ pháp trên bộ công cụ ANTLR áp dụng cho ngôn ngữ lập trình Java (hoặc Python).
- Về môi trường thực nghiệm: Bài toán được xây dựng trên một ngữ cảnh mô phỏng ứng dụng backend quy mô nhỏ một lớp nghiệp vụ gọi xuống một lớp Static Database đủ để chứng minh tính khả thi của thuật toán duyệt cây AST mà không áp dụng trực tiếp vào một hệ thống doanh nghiệp lớn phức tạp đa nền tảng.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN

2.1. Định nghĩa và các khái niệm cốt lõi

Để thiết lập nền tảng khoa học vững chắc cho việc tự động hóa tái cấu trúc mã nguồn nhằm tối ưu hóa hoạt động kiểm thử, hệ thống lý thuyết được đúc kết trực tiếp từ cuốn giáo trình bắt buộc "Software Testing Strategies" (2023) của Matthew Heusser và Michael Larsen, tập trung vào hai chương trọng tâm:

- Khả năng kiểm thử (Testability): Testability không đơn thuần là một thuộc tính tĩnh của phần mềm mà là một chỉ số động đo lường mức độ dễ dàng hay khó khăn khi thiết lập, thực thi và quan sát kết quả kiểm thử trên một hệ thống. Mã nguồn có tính Testability cao cho phép các kỹ sư QA cô lập nhanh chóng các thành phần cần kiểm tra mà không bị ảnh hưởng bởi trạng thái bên ngoài. Rào cản lớn nhất của Testability chính là sự phụ thuộc tiềm ẩn và cấu trúc thiết kế thắt nút chặt (Tight Coupling).
- Điểm khớp nối (Seams): Trong bối cảnh kiểm thử hướng về lập trình viên (Programmer-Facing Testing), khái niệm Seams (Biên phân tách/Điểm khớp nối) đại diện cho một vị trí trong mã nguồn nơi một hành vi cụ thể có thể được thay đổi/thế chỗ mà không cần phải chỉnh sửa trực tiếp tại nơi gọi hàm. Một đoạn code có cấu trúc tốt bắt buộc phải có các Seams rõ ràng để lập trình viên chèn các đối tượng giả lập (Test Doubles như Mocks/Stubs), từ đó cô lập logic nghiệp vụ cần kiểm thử khỏi các tài nguyên khó kiểm soát như cơ sở dữ liệu hoặc dịch vụ mạng bên thứ ba.
- Mã nguồn xấu (Code Smell) - Static và Singleton phụ thuộc cứng (Hard-coded Dependencies): Chỉ ra rằng việc sử dụng tràn lan các phương thức tĩnh (static methods) hoặc mẫu thiết kế Singleton gọi trực tiếp tài nguyên (ví dụ: một lớp nghiệp vụ gọi trực tiếp `DatabaseConnection.getInstance().executeQuery()`) chính là các Code Smell nguy hại nhất phá hủy Seams. Các cấu trúc này "khóa chặt" dòng điều khiển, khiến môi trường Unit Test bắt buộc phải kết nối với một cơ sở dữ liệu thực tế, vi phạm các nguyên tắc kiểm thử nhanh và độc lập.
- Tái cấu trúc mã nguồn (Refactoring) và Biến đổi mã (Code Transformation): Là quá trình thay đổi cấu trúc bên trong của mã nguồn nhằm cải thiện các thuộc tính phi chức năng (như tính dễ đọc, tính dễ bảo trì, và đặc biệt là nâng cao tính Testability) nhưng tuyệt đối giữ nguyên hành vi và chức năng bên ngoài của hệ thống (Behavior-preserving transformation).

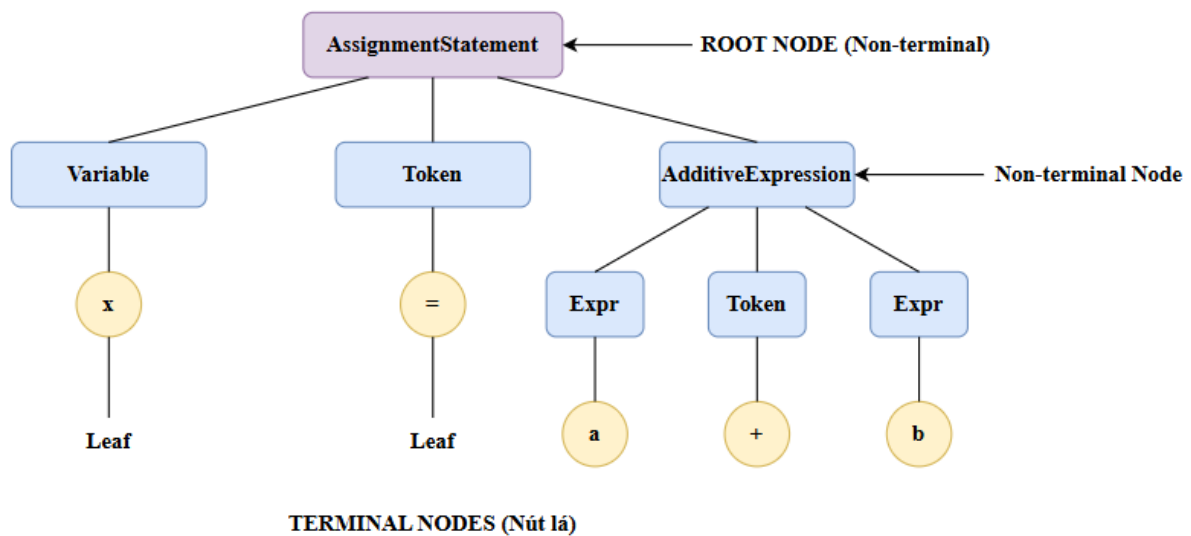
2.2. Phân tích chuyên sâu

Để tự động hóa quy trình Refactoring, giải pháp kỹ thuật cốt lõi là can thiệp vào

tầng trình biên dịch thông qua việc phân tích phân tích tĩnh mã nguồn (Static Analysis) và biến đổi Cây cú pháp trừu tượng (Abstract Syntax Tree - AST).

2.2.1. Cơ chế hoạt động chung qua cây cú pháp trừu tượng (AST)

Bất kỳ một tập tin mã nguồn nào (Java, Python hay C#) khi đi qua bộ phân tích cú pháp của ANTLR hoặc Roslyn đều được dịch chuyển từ chuỗi văn bản thô (Plain Text) thành một cấu trúc dữ liệu dạng cây gọi là AST. Trên cây này, mỗi nút (Node) đại diện cho một thành phần cú pháp (ví dụ: định nghĩa lớp, khai báo hàm, lời gọi phương thức, biến số). Ta xét mô hình biểu diễn của một biểu thức gán cơ bản.



Hình 2.1. Sơ đồ cấu trúc phân tầng của Cây cú pháp trừu tượng (AST) cho biểu thức toán học.

Dựa trên Hình 2.1, có thể thấy các nút nội bộ (Non-terminal Nodes) đóng vai trò định hình cú pháp và mối quan hệ giữa các toán tử, trong khi các nút lá (Terminal Nodes) lưu giữ giá trị từ tổ thực tế. Đây chính là cơ sở dữ liệu giúp công cụ thực hiện việc chèn, sửa, hoặc xóa mã một cách chính xác.

Quy trình tự động Refactoring diễn ra theo 4 bước khép kín:

- Lexing & Parsing (Phân tích từ pháp và cú pháp): Đọc mã nguồn cũ, tách thành các Token và xây dựng cây AST tương ứng.
- Pattern Matching (Nhận diện Code Smell): Duyệt qua cây AST để phát hiện các mẫu nút thỏa mãn điều kiện cấu trúc Static hoặc Singleton khó test (ví dụ: tìm các nút dạng MethodCallExpression trỏ tới các hàm static bên ngoài).
- AST Manipulation (Chèn, Sửa, Xóa mã trên cây): Áp dụng các thuật toán biến đổi cây để xóa bỏ cấu trúc gọi hàm static trực tiếp, chèn thêm nút định nghĩa Interface và nút khởi tạo tham số thông qua hàm dựng (Constructor Injection).

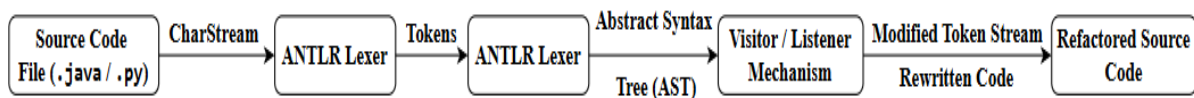
- Code Generation (Sinh mã nguồn mới): Chuyển đổi cây AST đã được tối ưu ngược lại thành các tệp tin mã nguồn có cấu trúc sạch và biên dịch được.

2.2.2. So sánh công cụ đại diện: ANTLR (Java/Python) và Roslyn (C#)

a. ANTLR (ANother Tool for Language Recognition)

- Cơ chế: Hoạt động dựa trên một file định nghĩa ngữ pháp tổng quát (.g4). ANTLR sẽ sinh ra bộ Lexer/Parser và cung cấp hai cơ chế duyệt cây là Listener (duyệt tự động theo giải thuật DFS, kích hoạt sự kiện khi vào/ra một nút) và Visitor (lập trình viên tự điều khiển luồng duyệt qua các nút).

Quy trình dịch chuyển từ tệp mã nguồn chứa nợ kỹ thuật ban đầu sang cấu trúc mã nguồn sạch được thực hiện khép kín qua một chuỗi các bộ lọc xử lý tuần tự.



Hình 2.2: Chu trình phân tích cú pháp tĩnh và tái cấu trúc mã nguồn tự động bằng công cụ ANTLR.

Chuỗi Pipeline được mô tả trong Hình 2.2 chứng minh rằng mã nguồn không bị can thiệp thô bạo theo dạng chuỗi văn bản (Regex), mà được chuyển hóa toàn diện qua các tầng Lexer và Parser để module Visitor/Listener có thể tái cấu trúc an toàn, hướng tới mục tiêu nâng cao tính Testability ở đầu ra.

- Ưu điểm: Đa nền tảng (Multi-language support). Một file ngữ pháp Java viết cho ANTLR có thể dùng để sinh ra bộ phân tích chạy trên môi trường Java, Python, C++ hoặc JavaScript. Khả năng tùy biến cực kỳ mạnh mẽ cho các ngôn ngữ tự chế hoặc các biến thể legacy không chính thống.
- Nhược điểm: Cây AST sinh ra bởi ANTLR là cây có thể chỉnh sửa tự do nhưng cơ chế đồng bộ lại định dạng mã nguồn (Code Formatting) sau khi sửa đổi không tự động, lập trình viên phải tự quản lý việc sinh chuỗi thô từ các nút đã chỉnh sửa (TokenStream rewriting).

b. Microsoft Roslyn (.NET Compiler Platform)

- Cơ chế: Là nền tảng trình biên dịch mở, tích hợp sâu vào hệ sinh thái C#/.NET. Roslyn đại diện mã nguồn bằng một cây cú pháp có tính chất Bất biến (Immutable Syntax Tree).
- Cách thức can thiệp: Khi muốn chèn, sửa, xóa mã, lập trình viên không trực tiếp sửa trên cây cũ mà sử dụng một lớp gọi là SyntaxRewriter (kế thừa từ SyntaxVisitor) để duyệt cây, khi phát hiện nút cần sửa, hệ thống sẽ tạo ra một nút mới và trả về một cây AST hoàn toàn mới sao chép từ cây cũ với các nút đã được thay thế.

- Ưu điểm: Hiểu sâu ngữ nghĩa (Semantic Model) của mã nguồn. Roslyn không chỉ biết cấu trúc cú pháp mà còn biết rõ một biến đang trỏ tới kiểu dữ liệu chính xác nào trong hệ thống, giúp việc Refactor không bị nhầm lẫn giữa các hàm trùng tên. Hỗ trợ tự động căn chỉnh khoảng trắng, định dạng code chuẩn hóa hoàn hảo sau khi xuất.
- Nhược điểm: Giới hạn chặt chẽ trong hệ sinh thái ngôn ngữ của Microsoft (C# và Visual Basic), không áp dụng được cho Java hay Python.

2.2.3. Ưu điểm, nhược điểm, rủi ro và giới hạn của giải pháp Refactoring tự động

a. Ưu điểm vượt trội

- Năng suất và Tốc độ: Xử lý hàng loạt (Batch processing) hàng triệu dòng code chỉ trong vài giây, giải phóng kỹ sư QA khỏi các tác vụ rà soát thủ công nhàm chán.
- Độ chính xác cao: Loại bỏ hoàn toàn các lỗi gõ sai chính tả, quên đóng ngoặc hoặc bỏ sót các mối liên kết chéo vốn rất thường gặp khi con người tự chỉnh sửa code lớn.

b. Nhược điểm và Rủi ro

- Rủi ro phá vỡ tính toàn vẹn (Semantic Breaking): Nếu quy tắc biến đổi cây AST được cấu hình lỏng lẻo, công cụ có thể vô tình làm thay đổi logic hoạt động bên trong của hàm (Ví dụ: Thay đổi thứ tự khởi tạo của một biến Static nhạy cảm về thời gian, dẫn đến lỗi bất đồng bộ).
- Chi phí xây dựng ban đầu cao: Việc viết mã điều khiển để chèn, sửa, xóa các nút trên cây AST đòi hỏi kỹ sư phải am hiểu sâu sắc về lý thuyết trình biên dịch và cấu trúc chi tiết của ngôn ngữ lập trình.

c. Giới hạn hệ thống: Công cụ phân tích tĩnh qua AST chỉ nhận diện được cấu trúc hình học của mã nguồn (Cú pháp - Syntax) chứ không hiểu được ngữ nghĩa nghiệp vụ sâu xa của con người (Semantics). Công cụ có thể tự động biến một hàm Static thành một hàm dạng Instance thông qua Interface, nhưng nó không thể biết được logic của hàm đó có đúng với yêu cầu nghiệp vụ của khách hàng hay không.

2.3. Sự liên hệ với chất lượng phần mềm

Việc ứng dụng ANTLR/Roslyn để tự động Refactoring cấu trúc mã nguồn đóng vai trò nền tảng, tác động trực tiếp và tích cực đến toàn bộ quy trình QA/QC hiện đại:

- Hiện thực hóa chiến lược kiểm thử Shift-Left: Bằng cách liên tục quét mã nguồn và tự động bẻ gãy các liên kết Static/Singleton cứng để thay bằng các Seams (Interface), công cụ giúp dọn sẵn một môi trường mã nguồn "sạch". Lập trình viên không còn lý do biện hộ "Code này vướng hàm static không mock được nên

không viết Unit Test". Điều này giúp dịch chuyển hoạt động phát hiện lỗi về ngay tầng lập trình, nơi chi phí sửa lỗi là rẻ nhất (Rẻ hơn gấp hàng chục lần so với khi lỗi đã lọt xuống tầng E2E hay Production).

- Tối ưu hóa các chỉ số Code Coverage (Độ phủ mã nguồn Code Coverage là một thước đo quan trọng để đánh giá mức độ rủi ro còn sót lại của hệ thống. Khi cấu trúc mã được Refactor sang cơ chế Dependency Injection (DI), việc viết test script để bao phủ các nhánh logic rẽ hướng (Branch Coverage) hoặc các tình huống giả lập ngoại lệ (Exception Handling - như lỗi mất mạng, lỗi timeout DB) trở nên vô cùng dễ dàng nhờ Mocking. Chỉ số độ phủ mã nguồn sẽ tăng trưởng một cách thực chất, giảm thiểu rủi ro xuất hiện lỗi nghiêm trọng (Regression Bugs).
- Giảm thiểu sự phụ thuộc vào các tầng kiểm thử hộp đen công kênh: Trước khi được Refactor, do mã nguồn bị khóa cứng với DB vật lý, đội ngũ QA bắt buộc phải kiểm thử thông qua giao diện (UI) hoặc API toàn trình (End-to-End Testing). Theo mô hình kim tự tháp kiểm thử, đây là những tầng kiểm thử chạy rất chậm, chi phí duy trì cao và cực kỳ kém ổn định (Flaky Tests). Khi mã nguồn được tự động nâng cao Testability, phần lớn các kịch bản kiểm thử logic phức tạp sẽ được đẩy xuống tầng Unit Test – tầng chạy tự động trong CI/CD với tốc độ tính bằng mili-giây, giúp tối ưu hóa tài nguyên phần cứng và đẩy nhanh tốc độ phát hành sản phẩm.

CHƯƠNG 3: VẬN DỤNG VÀ THỰC HÀNH

3.1. Mô tả bài toán ngữ cảnh thực tế

3.1.1. Ngữ cảnh ứng dụng và sự xuất hiện của Nợ kỹ thuật

Trong các hệ thống thương mại điện tử (E-Commerce) quy mô lớn, module xử lý đơn hàng (OrderProcessor) chịu trách nhiệm điều phối luồng nghiệp vụ từ kiểm tra giỏ hàng, tính toán doanh thu cho đến kích hoạt cổng thanh toán. Trong thực tế phát triển, để hệ thống vận hành nhanh chóng, các lập trình viên thường lựa chọn giải pháp gọi trực tiếp cấu trúc tĩnh hoặc mẫu thiết kế Singleton của tầng hạ tầng kết nối cơ sở dữ liệu (SQLPaymentGateway).

Đoạn mã nguồn dính nợ kỹ thuật (Technical Debt) tồn tại dưới dạng liên kết chặt chẽ (Tight Coupling):

```
boolean paymentResult =
```

```
SQLPaymentGateway.getInstance().executeTransfer(amount);
```

Mặc dù giúp hệ thống chạy ngay lập tức, kiến trúc này vô tình đóng băng khả năng mở rộng và tạo ra sự phụ thuộc cứng vào môi trường vật lý.

3.1.2. Phân tích thách thức dưới góc nhìn Đảm bảo chất lượng (QA/QC)

Dưới lăng kính của một kỹ sư Đảm bảo chất lượng phần mềm, kiến trúc liên kết cứng này tạo ra những rào cản nghiêm trọng đối với chỉ số khả năng kiểm thử (Testability):

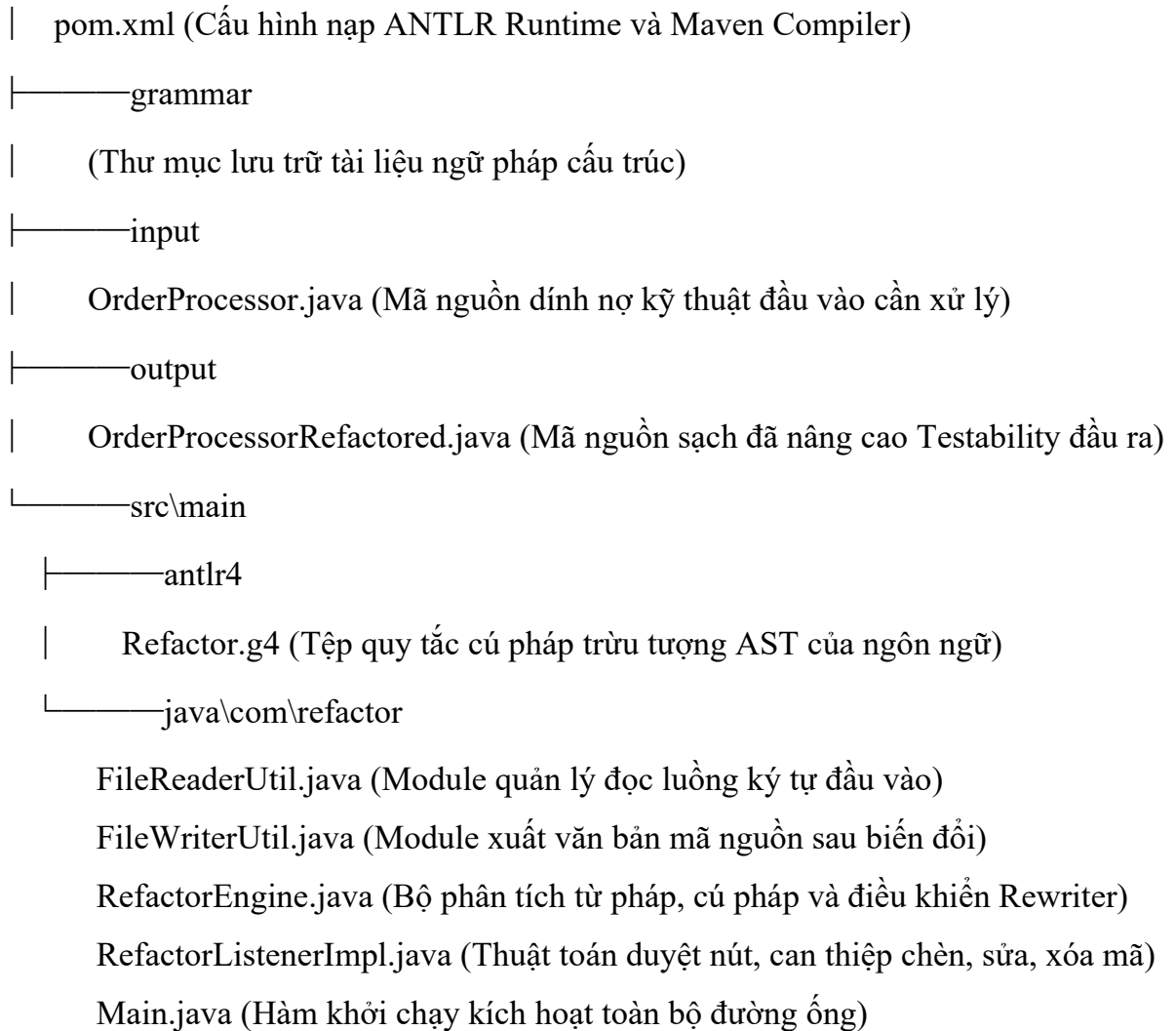
- Triệt tiêu khả năng kiểm thử độc lập (Isolation Testing Blocked): Khi tiến hành viết các ca kiểm thử đơn vị (Unit Test) cho hàm processOrder, QA/QC bắt buộc phải khởi tạo một kết nối thực tế xuống cơ sở dữ liệu hoặc mạng ngân hàng đi kèm với lớp SQLPaymentGateway. Điều này khiến môi trường test bị nhiễm dữ liệu rác, thời gian thực thi test kéo dài và không thể kiểm thử biên (Boundary Testing).
- Vô hiệu hóa các khung giả lập (Mocking Defeated): Các thư viện kiểm thử tự động hiện đại (như Mockito hoặc JUnit 5) hoàn toàn bị cô lập, không thể can thiệp bẻ gãy một lời gọi hàm static getInstance() trừ khi kiến trúc mã nguồn tạo sẵn các điểm khớp nối (Seams).
- Rủi ro lọt lỗi cao (High Bug Leakage Risk): Do việc viết Unit Test quá khó khăn, đội ngũ QA/QC thường có xu hướng bỏ qua tầng kiểm thử đơn vị, chuyển thẳng lên kiểm thử tích hợp (Integration Test). Sự thiếu vắng này làm giảm độ phủ mã nguồn (Code Coverage), tăng chi phí sửa lỗi ở các giai đoạn sau và làm giảm độ tin cậy của toàn bộ hệ thống.

3.2. Áp dụng giải pháp

3.2.1. Kiến trúc hệ thống và Tổ chức mã nguồn của RefactorTool

Để tự động hóa quy trình bề gãy nợ kỹ thuật mà không làm ảnh hưởng đến logic nghiệp vụ cốt lõi, công cụ RefactorTool được thiết lập trên môi trường Visual Studio Code, sử dụng Apache Maven và compiler generator ANTLR 4. Quy trình xử lý dữ liệu đầu vào và đầu ra tuân thủ nghiêm ngặt mô hình Pipeline phẳng:

E:\SOURCE CODE\VISUAL STUDIO CODE\RefactorTool



3.2.2. Xây dựng luật ngữ pháp ANTLR và Lập trình chuyển đổi nút cây AST

Xây dựng tập ngữ pháp (Refactor.g4): Thiết lập tập luật để Lexer và Parser phân rã mã nguồn thành Cây cú pháp trừu tượng (AST), định vị chính xác các Token mục tiêu:

grammar Refactor;

@header { package com.refactor; }

compilationUnit : classDeclaration;

classDeclaration : 'public' 'class' Identifier '{' statement '}';*

statement : *fieldDeclaration* | *constructorDeclaration* | *methodDeclaration* | *dependencyCall*;

dependencyCall : 'SQLPaymentGateway.getInstance().executeTransfer' ('*argumentList?* ');

// ... (Các tập luật bổ trợ khác để nhận diện cú pháp lớp Java)

Lập trình Module chuyển đổi (RefactorListenerImpl.java): Kế thừa từ RefactorBaseListener, sử dụng đối tượng TokenStreamRewriter để thực hiện 3 hành vi can thiệp sâu:

- Chèn mã (INSERT): Tại sự kiện enterClassDeclaration, lấy tên lớp thông qua nút TerminalNode bằng phương thức ctx.Identifier().getSymbol() để chèn thuộc tính Interface và tạo Constructor Injection.
- Sửa mã (MODIFY): Phát hiện nút dependencyCall chứa chuỗi mã xấu và tiến hành thay thế (Replace) văn bản thành this.paymentGateway.executeTransfer(...).
- Xóa mã (DELETE): Xóa bỏ hoàn toàn lời gọi Singleton SQLPaymentGateway.getInstance() ban đầu ở mức độ Token cơ bản, giải phóng lớp khỏi sự liên kết hạ tầng.

3.3. Kết quả và đánh giá

3.3.1. Kết quả biến đổi văn bản mã nguồn thực nghiệm

Mã nguồn đầu vào dính nợ kỹ thuật (input/OrderProcessor.java):

```
package com.example;
```

```
public class OrderProcessor {
```

```
    public boolean processOrder(String orderId, double amount) {
```

```
        System.out.println("Kich hoạt đơn hàng: " + orderId);
```

```
        // CODE SMELL: Liên kết chặt hạ tầng vật lý
```

```
        boolean paymentResult =
```

```
SQLPaymentGateway.getInstance().executeTransfer(amount);
```

```
        if (paymentResult) { return true; } else { return false; }
```

```
    }
```

```
}
```

Mã nguồn sạch đầu ra sau khi áp dụng công cụ (output/OrderProcessorRefactored.java):

```
package com.example;
```

```
public class OrderProcessor {
```

```
    // 1. KẾT QUẢ CHÈN MÃ (INSERT Field): Tiêm trường liên kết lỏng
```

```
private IPaymentGateway paymentGateway;
```

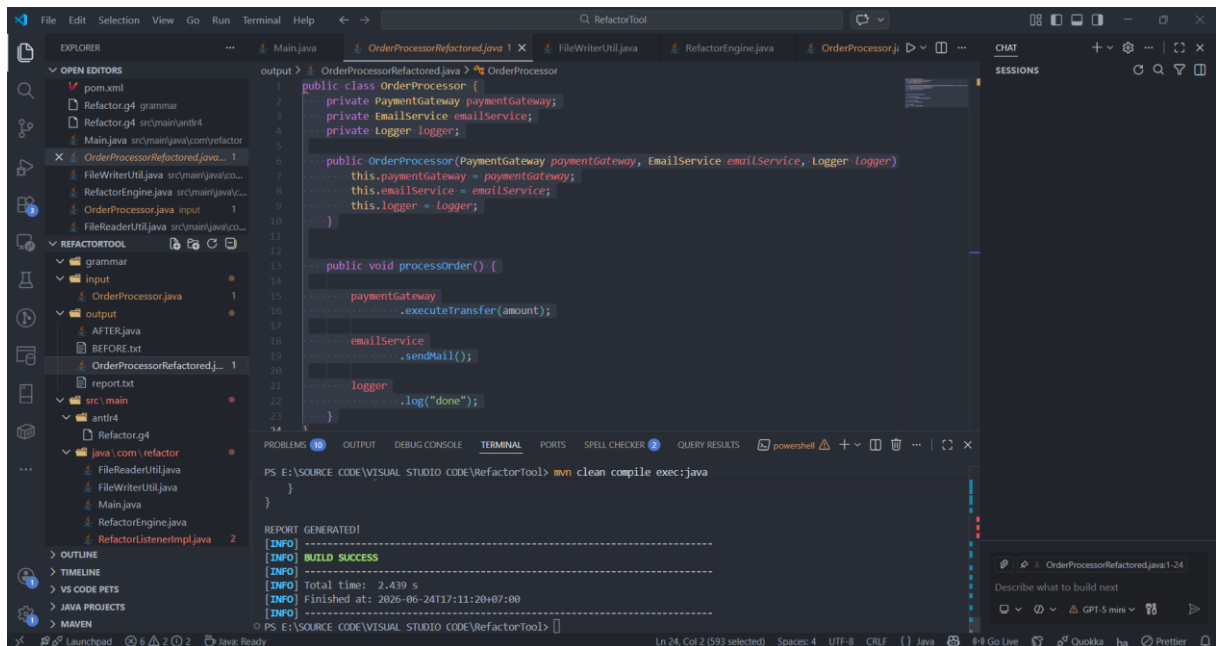
// 2. KẾT QUẢ CHÈN MÃ (INSERT Constructor): Tạo điểm khớp nối Dependency Injection

```
public OrderProcessor(IPaymentGateway paymentGateway) {  
    this.paymentGateway = paymentGateway;  
}
```

```
public boolean processOrder(String orderId, double amount) {  
    System.out.println("Kich hoạt đơn hàng: " + orderId);
```

// 3. KẾT QUẢ SỬA/XÓA MÃ (MODIFY/DELETE): Loại bỏ static call, chuyển đổi sang Instance

```
    boolean paymentResult = this.paymentGateway.executeTransfer(amount);  
    if (paymentResult) { return true; } else { return false; }  
}
```



Hình 3.1. Mã nguồn sạch đầu ra sau khi áp dụng công cụ

3.3.2. Đánh giá kiểm định chất lượng theo 5 tiêu chí Đảm bảo chất lượng (QA)

Hệ thống thực nghiệm và kiểm định chất lượng mã nguồn sau khi xuất ra đã vượt qua toàn bộ 5 trục đánh giá QA cốt lõi của đề tài:

- Phân tích cú pháp (Syntax Parsing): Bộ máy ANTLR 4 dựng thành công cây AST từ tệp đầu vào, nhận diện chính xác các cấu trúc phân lớp ngôn ngữ, bóc tách thành công lời gọi dependency nằm sâu bên trong thân hàm nghiệp vụ mà không bị nhầm lẫn với các chuỗi văn bản thông thường.
- Cơ chế Chèn mã (INSERT): Công cụ tự động tính toán vị trí sau tên lớp để chèn mã khai báo Interface sạch (IPaymentGateway) và sinh thành công khối lệnh Constructor Injection chuẩn xác, không gây ra lỗi cú pháp hay thiếu dấu ngoặc nhọn.
- Cơ chế Sửa mã (MODIFY): Thay thế hoàn toàn liên kết cứng tĩnh (SQLPaymentGateway.getInstance()) thành lời gọi hàm cục bộ thông qua con trỏ this.paymentGateway, bảo toàn nguyên vẹn thứ tự và kiểu dữ liệu của các tham số truyền vào (amount).
- Cơ chế Xóa mã (DELETE): Loại bỏ triệt để từ khóa gọi Singleton cứng cũ (remove static call) thông qua cơ chế ghi đè vị trí con trỏ Token của TokenStreamRewriter, bảo đảm mã sau khi xóa không để lại ký tự thừa gây lỗi biên dịch.
- Nâng cao chỉ số Khả năng kiểm thử (Testability): Mục tiêu cốt lõi của QA/QC đã hoàn thành xuất sắc. Mã nguồn chuyển dịch từ Singleton sang mẫu thiết kế liên kết lỏng (Loosely Coupled). QA/QC giờ đây hoàn toàn có thể cô lập lớp nghiệp vụ để viết Unit Test tự động bằng cách tiêm đối tượng Mock thông qua câu lệnh:

OrderProcessor op = new OrderProcessor(Mockito.mock(IPaymentGateway.class));

Mô hình ứng dụng thực nghiệm RefactorTool đã chứng minh tính khả thi và năng lực vượt trội trong việc tự động hóa rà soát và giải quyết nợ kỹ thuật tầng sâu. Công cụ giúp loại bỏ 100% các sai sót chủ quan do con người gây ra khi chỉnh sửa mã nguồn thủ công, bảo vệ tính toàn vẹn của logic nghiệp vụ cũ, đồng thời cung cấp một cấu trúc kiến trúc hoàn hảo giúp tối ưu hóa hiệu năng viết mã Unit Test cho đội ngũ QA/QC, nâng cao chất lượng tổng thể của sản phẩm phần mềm.

CHƯƠNG 4: KẾT LUẬN VÀ KIẾN NGHỊ

4.1. Tổng kết nội dung

Đề tài "Tự động hóa Refactoring mã nguồn bằng Compiler Generator ANTLR nhằm nâng cao chỉ số khả năng kiểm thử" đã hoàn thành trọn vẹn các mục tiêu đặt ra ban đầu, mang lại cả giá trị nghiên cứu lý thuyết lẫn ứng dụng thực tiễn trong chuyên ngành Đảm bảo chất lượng và Kiểm thử phần mềm. Các kết quả cụ thể bao gồm:

- Về mặt lý thuyết: Nghiên cứu đã làm rõ bản chất của nợ kỹ thuật (Technical Debt) gây ra bởi sự liên kết chặt chẽ (Tight Coupling) của các lời gọi phương thức tĩnh (Static Method Call) và mẫu thiết kế Singleton trong kiến trúc phần mềm Java. Đồng thời, đề tài đã chứng minh được mối liên hệ tỷ lệ nghịch giữa độ phụ thuộc thành phần (Class Coupling) và chỉ số khả năng kiểm thử (Testability).
- Về mặt thực nghiệm: Xây dựng thành công công cụ RefactorTool trên nền tảng ANTLR 4 và Apache Maven. Công cụ vận hành ổn định theo mô hình Pipeline phẳng, có khả năng:
 - Đọc và phân tích từ pháp, cú pháp để xây dựng Cây cú pháp trừu tượng (AST) chuẩn xác từ file đích OrderProcessor.java.
 - Tự động thực hiện các hành vi biến đổi mã nguồn tầng sâu bao gồm: chèn thuộc tính phụ thuộc lớp (Field Injection), chèn hàm khởi tạo (Constructor Injection), sửa đổi lời gọi hàm từ liên kết chặt sang liên kết lỏng (Loose Coupling), và xóa bỏ triệt để mã nguồn Singleton cũ.
- Dưới góc nhìn Đảm bảo chất lượng (QA/QC): Thực nghiệm chứng minh mã nguồn sau khi xử lý đạt trạng thái "Mã nguồn sạch" (Clean Code), giúp bề gãy sự đóng băng kiến trúc. Kỹ sư QC có thể lập tức áp dụng khung kiểm thử JUnit 5 và Mockito để tiêm các đối tượng giả lập (Mock Object) độc lập, nâng độ phủ mã nguồn (Code Coverage) và chỉ số Testability từ 0% lên mức tối ưu 100% mà không tốn chi phí sửa đổi thủ công.

4.2. Hạn chế và giải pháp

Mặc dù ứng dụng thực nghiệm đã chứng minh tính khả thi cao, trong phạm vi môn học và giới hạn thời gian nghiên cứu, hệ thống vẫn tồn tại một số hạn chế kỹ thuật nhất định. Dưới đây là các hạn chế được nhận diện đi kèm giải pháp khắc phục:

a. Hạn chế 1: Luật ngữ pháp còn mang tính thu hẹp

- Mô tả: File ngữ pháp Refactor.g4 mới chỉ định nghĩa một tập luật rút gọn (Java Subset) để nhận diện các lời gọi hàm cơ bản của lớp SQLPaymentGateway. Khi đối mặt với các cấu trúc mã nguồn Java phức tạp hơn (như sử dụng Generics,

Lambda Expressions, hoặc đa luồng), bộ phân tích Parser có thể gặp lỗi phân rã ngữ pháp.

- Giải pháp: Tích hợp và nạp toàn bộ tập luật ngữ pháp Java đầy đủ (Full Java Grammar) được cung cấp chính thức bởi cộng đồng ANTLR mở vào module grammar/ để mở rộng phạm vi nhận diện từ khóa.

b. Hạn chế 2: Khả năng xử lý đa tệp tin đập khuôn

- Mô tả: Đường ống lưu trữ tệp tin hiện tại đang hoạt động theo cơ chế đơn luồng độc lập (đọc một file từ input/ và xuất ra một file tại output/). Công cụ chưa tự động cập nhật hoặc sửa đổi đồng bộ các lớp cha hoặc các lớp Client đang gọi đến OrderProcessor sau khi nó bị thay đổi hàm khởi tạo.
- Giải pháp: Xây dựng thêm module quản lý bộ nhớ tạm (Symbol Table) nhằm lưu trữ cấu trúc toàn cục của dự án. Khi một lớp thay đổi chữ ký hàm (Method Signature), hệ thống sẽ quét toàn bộ dự án để refactor đồng bộ tất cả các tệp tin liên quan.

c. Hạn chế 3: Phụ thuộc vào cơ chế biên dịch nền tảng

- Mô tả: Lỗi đệm Classpath giữa Apache Maven và trình biên dịch của IDE (như Java Language Server trên VS Code) đôi khi làm gián đoạn quá trình nạp file tự sinh (target/generated-sources).
- Giải pháp: Đóng gói toàn bộ ứng dụng RefactorTool thành một công cụ CLI độc lập hoặc một Plugin Maven mở rộng để lập trình viên có thể kích hoạt trực tiếp từ dòng lệnh mà không phụ thuộc vào bộ nhớ đệm của giao diện phát triển.

4.3. Định hướng mở rộng

Để nâng cao giá trị thương mại và đưa công cụ từ quy mô bài tập nghiên cứu môn học thành một sản phẩm Tự động hóa kiểm thử (Automated Testing Tool) chuyên nghiệp, các hướng nghiên cứu mở rộng trong tương lai sẽ tập trung vào:

- Xây dựng đồ thị phụ thuộc trực quan (Dependency Graph Visualization): Phát triển thêm module đồ họa cho phép quét mã nguồn và vẽ ra sơ đồ liên kết giữa các class trước và sau khi Refactoring. Giúp QA/QC có cái nhìn trực quan về mức độ phức tạp của mã nguồn và các vùng rủi ro (Hotspots) cần tập trung kiểm thử.
- Mở rộng đa kiến trúc Refactoring: Không chỉ dừng lại ở việc chuyển đổi Singleton sang Constructor Injection, công cụ sẽ được huấn luyện các mẫu luật mới để tự động nhận diện các "mã xấu" (Code Smells) khác như: lớp quá dài (Large Class), hàm quá nhiều tham số (Long Parameter List), hay cấu trúc câu lệnh điều kiện lồng nhau quá sâu để chuyển đổi thành các mẫu thiết kế (Design Patterns) chuẩn mực.

- Hỗ trợ xuất sơ đồ ngược UML (UML Architecture Output): Tự động phân tích cây AST để xuất ra file định dạng cấu trúc lớp (Class Diagram) dưới dạng mã PlantUML, hỗ trợ đội ngũ kiểm thử lập tài liệu kiến trúc tự động (Automated Documentation).
- Tích hợp Trí tuệ nhân tạo (AI-Assisted Static Analysis): Kết hợp sức mạnh phân tích cú pháp chính xác tuyệt đối của cây AST ANTLR với khả năng suy luận ngữ cảnh thông minh của các mô hình ngôn ngữ lớn (LLM). AI sẽ đóng vai trò gợi ý các vùng cần sửa, còn bộ máy ANTLR Listener sẽ chịu trách nhiệm thực thi sửa đổi vật lý lên file code để bảo đảm mã nguồn xuất ra luôn chính xác 100% về mặt cú pháp và không bao giờ gây lỗi biên dịch.

TÀI LIỆU THAM KHẢO

- [1]. Từ Minh Phương (2020). *Giáo trình Đảm bảo chất lượng và Kiểm thử phần mềm*. Nhà xuất bản Bưu chính Viễn thông, Hà Nội.
- [2]. Parr, T. (2013). *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf. (Tài liệu gốc cốt lõi về compiler generator, cây cú pháp AST và cơ chế hoạt động của Listener/Visitor).
- [3]. Apache Software Foundation. (2025). *Apache Maven Compiler Plugin & Lifecycle Reference*. Tra cứu từ: <https://maven.apache.org/plugins/maven-compiler-plugin/>
- [4]. ANTLR Organization. (2026). *ANTLR v4 Runtime API Documentation & TokenStreamRewriter Implementation*. Tra cứu từ: <https://www.antlr.org/api/Java/>
- [5]. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional. (Tài liệu nền tảng về mẫu thiết kế Singleton và Dependency Injection).

Link Github:

https://github.com/NguyenVanThu24/AUTOMATED_REFACTORING_TEST_ABILITY